

Relatório de Desenvolvimento de Projeto: Cyberpunk ><#>

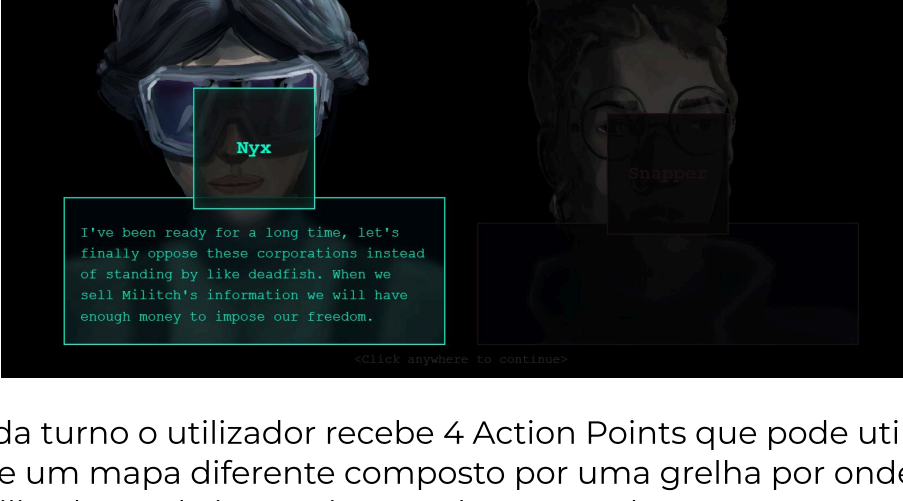
Contextualização - O projeto desenvolvido consiste numa experiência narrativa web com elementos de estratégia por turnos em perspetiva isométrica 3D, construída com recurso a HTML, CSS e JavaScript.

A narrativa insere-se no universo Cyberpunk RED localizada em Portugal. O jogador controla Nyx, uma Netrunner da facção "NetFish", num assalto ao porto de Leças (controlado pela corporação Militech) com o objetivo de roubar informação confidencial, vendê-la e libertar Matosinhos da dependência financeira das corporações.

A principal mecânica do jogo foca-se na dualidade entre Meatspace e Netspace. O jogador tem de atravessar o mundo físico (Meatspace) sem ser visto e infiltrar-se em "NET Architectures" (Netspace) através de terminais para atravessar obstáculos no mundo físico.

Introdução - A experiência começa com um Loading Screen (implementado para garantir que os aduções carregam antes de se começar o jogo) seguido pelo Main Menu com as opções "NEW GAME" e "LEVEL SELECTION" (onde se pode aceder a qualquer um dos níveis diretamente), ao premir a primeira opção começa a secção introdutória onde é apresentado o contexto inicial da história com recurso a um mapa de background, uma caixa de texto e overlays de zonas no mapa (ex: territórios da Militech e ASec) que surgem dinamicamente em sincronia com o texto, seguindo de um breve diálogo entre diferentes personagens antes de começar o tutorial.

Sistema de Diálogo - Implementámos um sistema de diálogo "back-and-forth" dividindo o ecrã ao centro em dois painéis, o painel da esquerda é utilizado para a Nyx enquanto que o da direita é usado para outras personagens, cada um dos painéis tem uma caixa de texto em frente de um portrait da personagem. Para tornar mais intuitivo quem está a falar adicionámos um efeito de escrita gradual ao texto e um efeito ao painel onde a personagem que não está a falar fica escurecida dando foco à atual.



Mecânicas - Em cada turno o utilizador recebe 4 Action Points que pode utilizar para se mover. Cada nível dispõe de um mapa diferente composto por uma grelha por onde entidades se podem mover. O utilizador pode interagir com elementos do mapa ou mover-se para células da grelha ao custo de 1 Action Point por célula de distância.

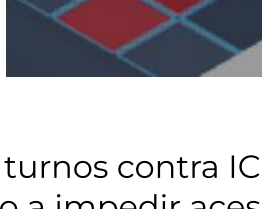
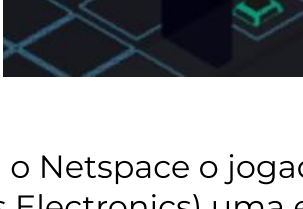
As interações estão divididas em dois espaços: o mundo real ou Meatspace e um mundo virtual ou Netspace que apenas pode ser acedido através de terminais permitindo a um netrunner entrar na "NET Architecture" dessa rede, essencialmente um elevador que para em vários andares, incluindo cada um outro nível.

Ambos os mundos estão interligados, podendo pensar no mundo virtual como um overlay por cima do mundo real, os corpos das personagens não desaparecem do mundo físico, elas apenas passam a ver o mundo virtual por cima do físico.



Meatspace - Em meatspace estão presentes diferentes tipos de obstáculos:

- guardas estáticos que rodam na sua posição ou guardas com rotas predefinidas, cada um com um cone de visão (representados por highlights vermelhos da grelha) que se atualizam dinamicamente, forçando o jogador a planejar o seu movimento para os evitar.
- Portas trancadas e câmaras de vigilância que requerem aceder a um terminal associado para serem abertas ou desativadas.
- Plataformas móveis em que o jogador pode subir para acompanhar o seu movimento.



Netspace e Combate - para o Netspace o jogador interage em combate de turnos contra ICE (Intrusion Countermeasures Electronics) uma espécie de antivírus destinado a impedir acessos indesejados, com o objetivo de chegar ao fundo de cada NET Architecture e desativar obstáculos em Meatspace.

De modo a combater os diferentes ICE Nyx está equipada com diferentes programas, estes abrangem entre opções defensivas, ofensivas ou até de utilidade para a permitir percorrer as NET Architectures:

Cada programa tem um custo de 1 Net Action, sendo que ações como mover de célula e descer ou subir de andar (usando ASCEND ou DIVE) também custam 1 Net Action, os programas têm os seguintes usos:

- SONAR.EXE revela outros andares da architecture e onde se encontram inimigos no stack do lado direito do ecrã.
- SWIM.EXE tem 30% de chance de fazer com que o jogador desça para o andar inferior perdendo a atenção de um ICE.
- HARPOON.EXE permite atacar um ICE à distância.
- SWORDFISH.EXE ataca um ICE adjacente.
- SCALES.EXE dá ao jogador 70% de chance de negar as próximas 2 instancias de dano.

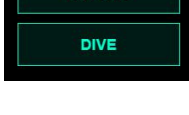
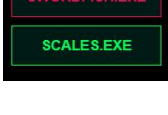
Cada ICE um tem um aspeto diferente e um efeito especial:

- Hellhound faz com o jogador leve dano passivamente nos próximos turnos.
- Wisp reduz o máximo de Net Actions do próximo turno para 1.
- Asp escolhe 1 das habilidades para de combate para a desativar por 3 turnos sempre que ataca (apenas mantendo 1 habilidade desativada de cada vez).
- Scorpion impede o movimento entre células do jogador.
- Kraken impede que o jogador fuja para outro andar ou use SWIM.EXE.



Adicionámos também efeitos visuais e sonoros para quando a Nyx leva dano bem como uma sequência visual quando fica sem vida.

UI Netspace - Para o UI dentro de Netspace, do lado direito utilizamos uma representação visual (em stack) dos andares da Architecture atual, sendo o andar atual saliente e andares com inimigos ativos dispostos a vermelho. No lado esquerdo estão presentes os botões para as diferentes ações que a Nyx pode tomar, seguindo um código de cor com vermelho para ataques, verde para defesa e azul para funções de movimento e visão.



Modelação e Animação 3D - No decorrer do projeto este evoluiu de geometria básica que tínhamos na primeira meta para cenários e modelos 3D feitos em blender.

Todos os modelos foram desenvolvidos em Blender (com recurso a imports do sketchfab e blenderkit apesar de terem sido imensamente editados e simplificados) e exportados no formato GLB. A animação é gerida através de AnimationMixer e SkeletonUtils, permitindo que os modelos dêem play a animações feitas com armatures.

Alguns das animações foram feitas exclusivamente em blender (como a Idle animation da Nyx) enquanto que outras foram feitas dentro da função animate (como a inclinar e movimento do HARPOON.EXE visto que era uma animação dependente das coordenadas do inimigo) enquanto que outras foram feitas em mistura (o movimento do SWORD.EXE foi feito em blender enquanto que o fade out e fade in foram feitos por alteração da opacidade).

Sistema de Transparência Dinâmica - Para garantir que o jogador nunca perde a visibilidade da personagem em perspetiva isométrica, implementámos código dentro da função animate que deteta em que objeto o rato está acima e se este é uma parede e não chão. Estas paredes tornam-se semitransparentes dinamicamente através da alteração da opacidade do material, voltando ao estado original assim que o jogador retirar o rato de cima delas.



Inclinação das cells por Raycasting - Dado que previ que fossem haver várias alterações ao modelos no seu desenvolvimento desenvolvi um sistema de raycasting de modo a permitir cenários com diferentes elevações (rampas e plataformas), ao invés de alterar cada tile à mão. O sistema utiliza raycasting para calcular a altura e a inclinação de cada tile da grelha tendo em conta a sua posição na grelha de map dentro de LEVEL_DATA, permitindo que os tiles, a personagem e os cones de visão dos guardas se adaptem perfeitamente ao relevo do terreno.



Problemas Encontrados e Soluções Desenvolvidas

Problema 1 - Alta complexidade de mecânicas - mesmo após simplificarmos as mecânicas de Netrunning presentes em Cyberpunk RED, quando fizemos playtesting na fase de teste os jogadores ficavam confusos em relação ao funcionamento do jogo.

Deste modo implementámos os seguintes sistemas:

Tutoriais - Dado a as mecânicas complexas, especialmente a transição para o Netspace e a sobrecarga súbita de informação decidimos implementar um nível de tutorial e tutoriais no início dos níveis que achámos necessário.

O nível de tutorial apresenta todas as mecânicas essenciais do jogo à medida que vão sendo apresentadas ou utilizadas, ajudando o jogador a interiorizar o que fazer e removendo parte do elemento de sobrecarga de informação do problema.

Sistema de Logs - De modo a dar mais informação sobre o que está a ocorrer no jogo (como os efeitos de cada ICE) implementámos um log de ações que pode ser expandido ao premir na sua zona para ver uma lista de mensagens anteriores, as duas mensagens anteriores são mostradas com opacidade reduzida despendo que não esteja expandido. Adicionalmente as mensagens têm a sua cor reduzida dependendo se são mensagens relacionadas com Netspace ou Meatspace de modo a tornar o sistema mais claro.

Problema 2 - Movimento repetitivo e confuso - inicialmente tínhamos implementado restrições para o movimento do jogador no qual apenas era possível mover para células adjacentes, adicionalmente quando o jogador ficava sem Action Points era preciso clicar no botão de fim de turno com o rato sem apresentar nenhum feedback visual exceto o contador de Ation Points no topo do ecrã.

Além disto no sistema que tínhamos inicialmente quando o jogador era apanhado era enviado para o início do nível, e quando ficava sem "vida" (representado pela barra de Neural integrity) dentro do Netspace voltava para o menu inicial após um delay sem feedback.

Tendo isto em conta fizemos as seguintes alterações:

Pathfinding - Implementação de um algoritmo de pathfinding (breadth-first search). O jogador pode agora colocar o rato sobre o destino desejado, o jogo calcula a rota, ilumina o caminho se for possível com base nos AP disponíveis e move a personagem de cell em cell (verificando se não entra no cone de visão de um guarda) até ao destino com um único clique.

Fim de turno automático - Caso o jogador meta o rato sobre uma célula para mover a personagem e não tiver AP a rota fica iluminada de laranja ao invés de ciano, ao premir numa dessas cells o turno do jogado é terminado automaticamente, dando trigger às alterações no cenário, e a personagem faz o movimento indicado.



Sistema de Checkpoints - Agora ao sempre que o jogador sai de um terminal essa posição na grelha é guardada para que o jogador lá volte caso seja apanhado por um guarda, utilizando o contexto de que o caminho que a Nyx tinha acabado de fazer onde foi apanhada era apenas um cálculo ou simulação do percurso e não algo real, sendo uma mensagem com esse contexto enviada para o log.

Problema 3 - O quarto nível - Apesar de inicialmente termos planeado 5 níveis sem contar com o tutorial acabámos por ter de remover o quarto nível "testing grounds" devido a:

- restrições de tempo,
- tamanho desproporcionalmente grande do quarto nível em relação aos restantes,
- falta de propósito das mecânicas de drones (efetivamente guardas que podem ser desligados) dado que eram uma combinação de guardas e câmaras e podiam ser substituídos por tais.

Acabámos por remover o 4º nível e os drones (apesar de haver uma implementação em código e estarem funcionais).

Trabalho realizado por cada membro

Diogo Botelho (2022213539) - Documentação, desenvolvimento do código, modelação e texturas, animação, áudio, diálogos, media de tutoriais, disponibilização no servidor do DEI

Gonçalo Albuquerque (2023206978) - Design da Nyx, diálogos, ilustrações